

Hyperparameter Tuning with Grid Search Cross-Validation

1. What You'll Learn in This Section

In this lesson, you'll learn to:

- Distinguish trainable parameters from hyperparameters and explain why each needs a different search strategy.
- Explain why learning rate and tree depth need careful tuning, using the overshoot and overfitting problems as evidence.
- Apply `GridSearchCV` to search a parameter grid jointly instead of tuning values one at a time.
- Read grid search output (`best_params_`, `best_score_`, `cv_results_`) and interpret a cross-validation-accuracy heatmap.

2. Detailed Explanation

Trainable parameters versus hyperparameters

A trainable (learning) parameter is a value that a model learns automatically from data during training. For a linear-style function $f(x) = w_0 + w_1 * x$, the bias term `w0` and the feature weights `w1`, `w2`, `w3`, ... are trainable parameters. Their optimal values come from running the training algorithm on `x_train` and `y_train`. Training itself is the search for the weight values that minimize error.

A hyperparameter, by contrast, is not learned from data at all — it is set externally, before or around training. Examples you'll meet throughout this topic:

- Maximum depth of a decision tree — how many levels deep the tree is allowed to grow.
- Minimum samples per leaf — the minimum number of training instances a node must hold before it can become a leaf, so a leaf doesn't get created for just one or two instances.
- Learning rate (`eta` or `alpha`) — shows up in almost every learning algorithm: logistic regression, linear regression, SVM, neural networks, transformers, and decision-tree-based boosting.
- Number of estimators — for ensemble methods like random forest, how many individual trees get built before their predictions are combined.
- Number of hidden layers — for neural networks, how many hidden layers the network has.

The distinction matters in practice: trainable parameters fall out automatically as a byproduct of running the training algorithm, while hyperparameters have to be found through some external search process — which is exactly what this topic is about.

Why hyperparameter values matter: learning rate and tree depth

Picture the error of a model — mean squared error for regression, cross-entropy loss for classification — plotted against one trainable weight like `w0`. That plot forms a curve with a minimum point, and training starts from a random position on the curve and moves toward that minimum. The learning rate controls how big each step toward the minimum is:

- A very high learning rate (an extreme example is `alpha = 10`) causes large jumps that overshoot the minimum repeatedly. The model may never settle near the optimal weight and can end up with high bias — performing poorly even on training data.

- A very low learning rate (an extreme example is `alpha = 0.01`) creeps toward the minimum very slowly. It eventually finds the optimal value, but wastes computational time getting there.

A balanced learning rate — not too small, not too large — is needed, and finding that balance is precisely the kind of problem hyperparameter tuning solves.

Tree depth causes a related but different issue. Left unrestricted, a decision tree keeps splitting until it correctly classifies every training instance, potentially down to leaves that classify only one or two instances each. For a dataset of 300 instances, a tree of depth 10 has up to 2^{10} possible leaf nodes — far more capacity than 300 instances justify. A very deep tree is a nested set of if-else rules fitted tightly to the training instances. It can score close to 100% on training data but fails to generalize, because unseen test instances rarely satisfy the same long chain of nested conditions. This pattern — strong training performance, weak test performance — is called high variance, also known as overfitting. A model around 95% accurate on training data that generalizes to, say, 85% test accuracy is preferable to one that hits 100% training accuracy but only 65% test accuracy.

Watch out for: chasing the highest possible training accuracy. The goal isn't a perfect score on training data — it's a model that holds up on data it hasn't seen. Limiting maximum depth and requiring a minimum number of samples per leaf are the two levers used to control this, but the right depth limit (four, five, six?) isn't something you can guess — it has to be found through tuning.

The trouble with tuning one hyperparameter at a time

A naive strategy: pick one hyperparameter, try several values, keep the best, then move to the next hyperparameter and repeat — treating each search as independent.

Worked example: for learning rate `eta`, five candidate values were tried — 0.1, 0.4, 0.6, 0.8, 0.9. For tree depth, five candidate values were tried — 2, 3, 4, 5, 6. Testing these sequentially costs $5 + 5 = 10$ separate training runs. Whichever `eta` performs best (0.6, in this example) gets fixed, and depth is then tuned separately against that fixed `eta`.

This has two problems:

1. Implementation cost — many independent runs are needed even for just two hyperparameters, and the run count grows further as more hyperparameters are added.
2. Non-optimal combinations — the value that wins for one hyperparameter in isolation isn't necessarily the best value once paired with the best value of another hyperparameter. If the true optimal depth were shallow, a slower learning rate might actually pair better with that shallow depth than the learning rate that looked "best" when depth was left at its default. Hyperparameters interact, so they need to be searched jointly, not independently.

Grid search: searching every combination together

Grid search is a general parameter-optimization strategy — not specific to machine learning, borrowed from domains like general computer algorithms and physics — that evaluates every combination of candidate hyperparameter values together, systematically, instead of tuning one at a time.

Worked whiteboard example with two hyperparameters:

- `eta` candidate values: 0.1, 0.3, 0.5, 0.6 (4 values)
- `depth` candidate values: 3, 4, 5 (3 values)

This produces a 4×3 grid of 12 total combinations. For every `(eta, depth)` cell, the same algorithm is trained and its cross-validation score is recorded in that cell. Once

every cell is filled, the combination with the highest cross-validation score is picked — in this example, `eta = 0.3, depth = 4` won. For two hyperparameters, the grid can be pictured as a 2D table. With three or more, the same logic applies, but the values get organized as a table instead of a picture — one hyperparameter repeated across blocks of rows. A grid beyond two dimensions can't be drawn on a flat screen.

The full set of candidate combinations being evaluated is called the search space. Cross-validation — rather than a single train/test split — is used to score each grid cell so the reported performance for each combination is reliable, not the result of one lucky or unlucky split.

Cross-validation and the role of K (number of folds)

Cross-validation splits the training data into multiple partitions, called folds, so that instead of relying on a single train/test split, each partition takes a turn as validation data while the rest is used for training. The final model produced this way is then evaluated once more on a completely separate, untouched test set. With K folds, the algorithm is trained and validated K times, and the mean of those K scores is reported as the cross-validation score.

K is chosen up front, but it is not tuned via grid search — varying K doesn't fundamentally change the outcome, only how finely the data is partitioned. The practical guidance is to size K to the dataset. A larger dataset (around 1,000,000 instances) can comfortably support a higher K , such as 10. A smaller dataset (around 5,000 instances) is better served by a lower K , such as 5, so each fold still holds a reasonable amount of data.

Two loss functions come up when scoring models during training: Mean Squared Error (MSE) and Mean Absolute Error (MAE) for regression, and cross-entropy loss (also called log-likelihood) for classification. Grid search and hyperparameter tuning apply

equally to classification and regression — only the internal loss function changes, not the tuning mechanism.

Worked example: predicting ad clicks

A synthetic dataset of 300 instances predicts whether a person clicks an advertisement (`clicked = 1`) or not (`clicked = 0`), using these features:

Feature	Description	Range
Ad position	Where the advertisement is placed on the screen	0 to 1
Time of day	Hour of the day	0 to 24
Category match score	How closely the ad matches the viewer's product interests	0 to 1
Image quality	How clear/sharp the ad image is	0 to 1
Noise	Random noise added to make the data more realistic	0 to 100

These features reflect everyday advertisement behavior:

- People click more during idle hours (for example, around 11 p.m.) than during working hours.
- They click more on ads matching products they already buy or browse.

- Ad position matters because most people are right-handed, so ads on the right, near the natural resting position for the hand, get more clicks.
- A blurry image reduces clicks because it feels spammy or low-effort even when the product itself is appealing.

A formula combines these features into a raw score, giving positive weight to favorable ad position, category match, and image quality, a negative-leaning term for time of day relative to working hours, and a bias term. That score passes through a sigmoid function to become `probability_of_click`, a value between 0 and 1. To turn this probability into an actual 0/1 outcome, it's compared against a separately generated random number: if the random number is lower than the probability of click, the outcome is recorded as 1; otherwise 0. This mirrors how real-world labels are usually provided directly rather than generated from a formula. The features and generated `clicked` outcome are assembled into a pandas DataFrame, split into `x` (feature columns) and `y` (the `clicked` column), and finally into `x_train`, `x_test`, `y_train`, and `y_test` using `train_test_split`. The mean of `y` across all 300 instances is approximately 0.41 — about 41% of instances result in a click.

A `DecisionTreeClassifier` trained with no hyperparameter tuning is allowed to grow to whatever depth it needs. Calling `get_depth()` on this default tree shows it grows to depth 10 for the 300-instance dataset — up to 2^{10} possible leaf nodes against only 300 training instances, far more capacity than the data justifies. This untuned tree achieves a test accuracy of 0.667 (66.7%), low for what is otherwise a straightforward classification task — the overfitting problem in action.

Running GridSearchCV and reading the results

The relevant imports:

```
import numpy as np
import pandas as pd
```

```
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

`GridSearchCV` is a scikit-learn class used directly, not implemented from scratch. It works with any estimator that has hyperparameters – decision trees, logistic regression, linear regression, and more – not only decision trees. A decision tree is used here specifically because it has multiple interacting hyperparameters, making it a good demonstration case.

The candidate values are defined as a `param_grid`, the dictionary of candidate hyperparameter values passed to `GridSearchCV`:

```
param_grid = {
    "max_depth": [2, 4, 6],
    "min_samples_leaf": [1, 5, 10]
}
```

`GridSearchCV` is then constructed and fit:

```
grid = GridSearchCV(DecisionTreeClassifier(), param_grid, cv=5)
grid.fit(X_train, y_train)
```

The first argument is the estimator to tune, the second is the parameter grid, and `cv=5` requests 5-fold cross-validation. With 3 candidate values for `max_depth` and 3 for `min_samples_leaf`, this produces a 3×3 grid of 9 total combinations. Each combination is evaluated across all 5 folds, and the mean of the 5 fold scores becomes that combination's cross-validation accuracy.

The fitted `grid` object exposes three key attributes:

- `best_params_` — the hyperparameter combination with the best cross-validation score. Here: `max_depth = 6, min_samples_leaf = 10`.
- `best_score_` — the best mean cross-validation accuracy achieved: 0.756, rounded to three decimal places.
- `cv_results_` — the full results for every combination tried, loadable into a `DataFrame` with columns `max_depth`, `min_samples_leaf`, `mean_test_score`, and `std_test_score`.

Sorting that results `DataFrame` by descending mean cross-validation accuracy puts the winner first:

<code>max_depth</code>	<code>min_samples_leaf</code>	mean CV accuracy
6	10	0.756
4	10	(second highest, close to 0.756)
6	1	(third highest)
4	5	0.711
...	...	...

Even though `max_depth = 6, min_samples_leaf = 10` wins outright, `max_depth = 4, min_samples_leaf = 5` scores 0.711 — about a 4.5-point gap. That makes a slightly

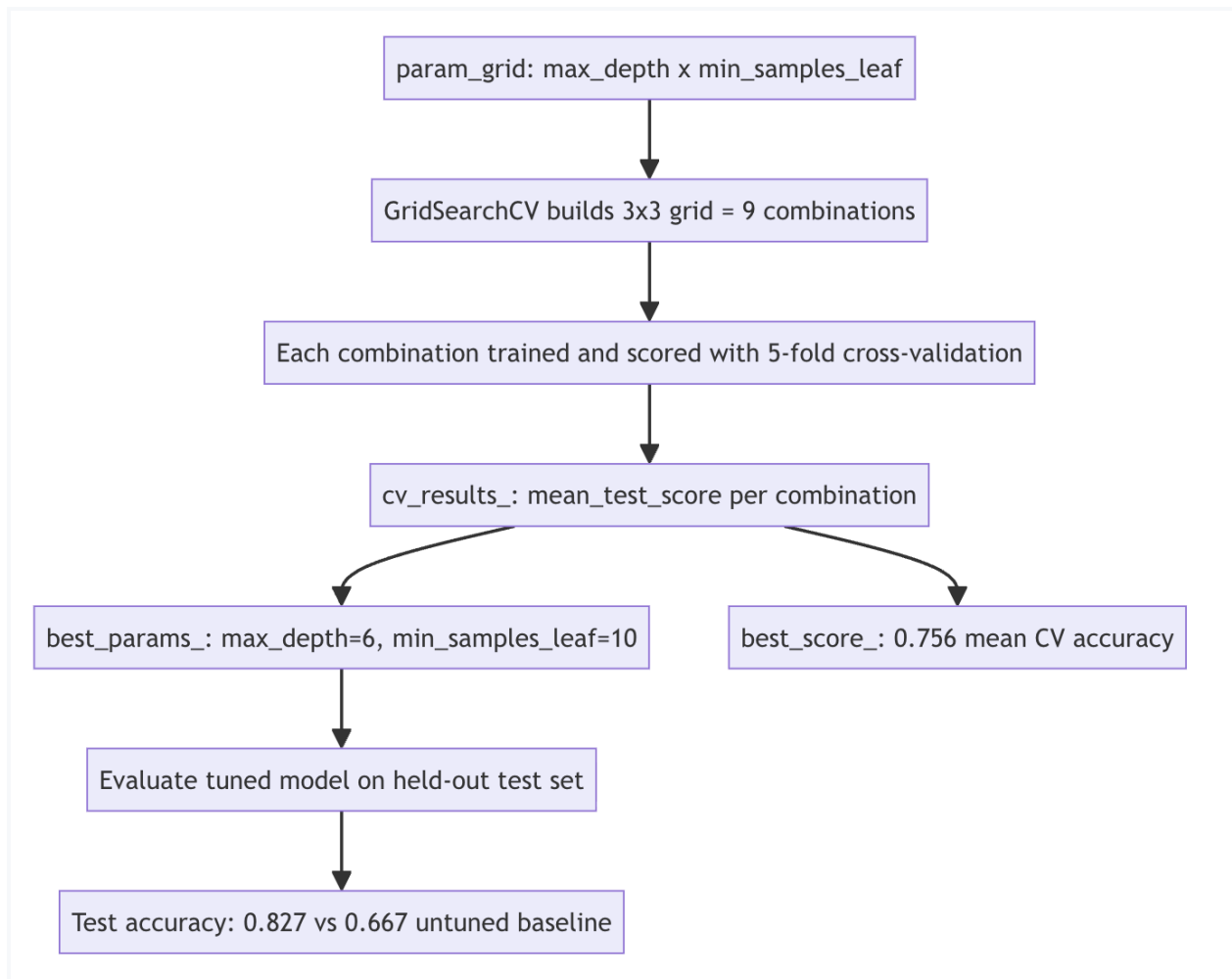
less complex tree a reasonable deployment trade-off if reducing execution complexity matters, though the default practice is to deploy the maximum-performing combination.

The results DataFrame can also be reshaped into matrix form with pandas' `pivot` function — `max_depth` as the row index, `min_samples_leaf` as the columns, and `mean_test_score` as the cell values:

```
pivot_table = results_df.pivot(  
    index="max_depth",  
    columns="min_samples_leaf",  
    values="mean_test_score"  
)
```

That pivoted matrix is displayed as a heatmap with `imshow`. The heatmap includes a color bar labeled "CV accuracy" (darker green for higher scores, lighter green for lower), axis tick labels for `max_depth` and `min_samples_leaf`, and a title, "Mean CV accuracy across the grid". Each cell's numeric value is overlaid at its center, rounded to three decimal places. The heatmap confirms visually that `max_depth = 6`, `min_samples_leaf = 10` is the brightest cell.

Here's the whole flow from parameter grid to final decision:



Confirming the improvement on test data

To confirm the grid search improvement is real, and not just an artifact of cross-validation, the tuned model's predictions on the held-out test set are compared against the untuned default tree's predictions on that same test set:

```
best_model_accuracy = accuracy_score(y_test, grid.predict(X_test))
```

Results on the test set:

- Default (untuned) decision tree: 0.667 (66.7%)

- Grid-search-tuned decision tree (`max_depth = 6, min_samples_leaf = 10`): 0.827 (82.7%)

Moving from 66.7% to 82.7% is a substantial improvement, showing that systematic hyperparameter tuning can produce a drastic, not marginal, performance gain. The cross-validation results tell the same story: the tuned model's `best_score_` of 0.756 was already the top-scoring cell in the grid, so the improvement shows up in cross-validation as well as on the held-out test set.

It's worth distinguishing between two kinds of numbers here. `grid.best_score_` (0.756) is a mean cross-validation score, computed by training and validating across 5 folds of the training data. In contrast, the 0.667/0.827 comparison uses a single held-out test set. Both are valid, but they measure different things and aren't expected to match exactly.

Extending to more hyperparameters, and pairing grid search with feature selection

The same grid search logic scales beyond two hyperparameters. Adding a third hyperparameter, such as learning rate, with two candidate values means the original 9 combinations of `max_depth` and `min_samples_leaf` get repeated once per learning-rate value — producing 18 rows total (9×2). The winning combination is still whichever row scores best on cross-validation. Grid search never tunes each hyperparameter alone and combines the individual "winners" afterward — every hyperparameter is searched jointly with every other hyperparameter, all together, in one grid.

Grid search (`GridSearchCV`) can also be paired with feature selection (`SelectKBest`) — an independent mechanism that ranks and selects the most significant features for prediction. `SelectKBest` keeps the best subset of features (for example, the best 5 or 10 out of 20) based on a significance score, reducing the feature space and computational complexity. It's typically applied first, during pre-processing, before

training begins. `GridSearchCV` tunes model hyperparameters (like tree depth and minimum samples per leaf) and runs as part of training itself. The two operate at different stages and address different aspects of the model — which features are used versus how the model is configured. Combining them can produce a better model than using either alone.

The resource-cost trade-off of grid search

Grid search costs more compute than training a single fixed-hyperparameter model, because it trains and cross-validates once per grid combination — and that cost grows quickly as more hyperparameters get added. This is one reason algorithms with many hyperparameters, such as random forests (number of estimators) and neural networks (number of hidden layers), take longer to tune than simpler algorithms.

This cost is acceptable because tuning happens once, during training. Once the optimal combination is found and a final model delivered, that model gets reused for prediction indefinitely, until requirements or the underlying data change, without repeating the search. Large pretrained models such as Gemini and ChatGPT/GPT (including versions like GPT-3 and GPT-4) follow the same logic at a much larger scale. They're extremely costly to train, involving billions of trainable parameters and many hyperparameters, and are trained once by organizations with large amounts of compute. Once trained, they get reused repeatedly — including by other teams building smaller models on top of them — rather than retrained from scratch each time.

Some search cost can be trimmed by using theoretical background to narrow the candidate range before running grid search, instead of searching an unconstrained range blindly — for example, a learning rate typically shouldn't go far below 0.1 or above roughly 1–2. Even so, the exact optimal value within a sensible range still has to be found through grid search rather than guessed.

3. Key Takeaways

- Trainable parameters are learned automatically from data during training; hyperparameters are set externally and must be found through a search process.
- Poorly chosen hyperparameters have real consequences: a learning rate that's too high overshoots the minimum, and one that's too low wastes time.
- An unrestricted tree depth causes overfitting (high variance) — great training performance but poor test performance.
- Tuning hyperparameters one at a time is flawed because it costs more runs overall and can miss the truly optimal combination, since hyperparameters interact with each other.
- Grid search fixes this by evaluating every combination jointly via cross-validation and picking the best-scoring cell.
- On the ad-click dataset, this took test accuracy from 0.667 (untuned) to 0.827 (tuned with `max_depth = 6, min_samples_leaf = 10`).
- The number of cross-validation folds (k) is chosen based on dataset size, not tuned via grid search, and grid search pairs well with independent techniques like `SelectKBest` feature selection.

Mental model: Think of grid search as testing every combination on a multiplication table at once — instead of picking the best row, then separately picking the best column, you fill in the whole table and pick the single brightest cell, because the best row and the best column don't always intersect at the best cell.